Soil Moisture Downscaling

Mayur Kunnitigi Matada

April 2026

Contents

Abstract 4

Executive Summary 5

1 Project Motivation and Scope 6
- 1.1 Why Spain Was Chosen . . . 6
- 1.2 Problem Definition . . . 6
- 1.3 Scope Boundaries . . . 7

2 Data Sources and Provenance 8
- 2.1 In Situ Labels: ISMN REMEDHUS . . . 8
- 2.2 Satellite and Reanalysis Predictors . . . 8
- 2.3 Quality Constraints and Alignment Logic . . . 9

3 Study Area, Temporal Scope, and Sampling Geometry 10
- 3.1 Baseline Scope . . . 10
- 3.2 Final Scope . . . 10
- 3.3 Patch-Centric Sampling . . . 11

4 Baseline Pipeline (get_spain_data.ipynb) 12
- 4.1 Baseline Role in the Project . . . 12
- 4.2 ISMN Parsing and Label Construction . . . 12
- 4.3 Feature Sampling from Earth Engine . . . 14
- 4.4 Merge and Sanity Diagnostics . . . 15
- 4.5 Baseline Residual XGBoost Modeling . . . 15
- 4.6 Baseline Outcome . . . 16

5 Final Pipeline (CNN_Spain.ipynb) 17
- 5.1 Design Rationale . . . 17
- 5.2 Long-Range Label Preparation . . . 17
- 5.3 Dynamic, Meteorological, and Static Feature Stack . . . 18
  - 5.3.1 Derived Variables . . . 18

5.3.2 Patch Tensor Construction . . . 18
5.4 TFRecord Export and PyTorch Ingestion . . . 19
5.5 Normalization and Seasonal Channels . . . 20

6 Model Architecture: MultiModalSoilNet **21**
6.1 Overview . . . 21
6.2 Branch A: Dynamic Spatial CNN . . . 21
6.3 Branch B: Static Center MLP . . . 22
6.4 Branch C: Temporal LSTM . . . 22
6.5 Fusion and Output Head . . . 23

7 Sequence Construction and Data Contracts **25**
7.1 Tensor Contracts . . . 25
7.2 Station-Aware Chronological Sequence Builder . . . 25

8 Training Objective, Optimization, and Tuning **27**
8.1 Robust Weighted Huber Loss . . . 27
8.2 Outlier Trimming by Quantiles . . . 27
8.3 Optimization Stack . . . 27
8.4 Hyperparameter Search Program . . . 29

9 Evaluation Protocol and Scientific Rigor **30**
9.1 Primary vs Secondary Split . . . 30
9.2 Metrics . . . 30
9.3 Calibration Layer . . . 31

10 Results **32**
10.1 Baseline Results . . . 32
10.2 Final Model Results . . . 32
10.3 Interpretation . . . 32
10.4 Comparative Summary . . . 33

11 Detailed Discussion **34**
11.1 Why Residual Learning Helps . . . 34
11.2 Why Explicit Routing Helps . . . 34
11.3 Why Station Split Is Non-Negotiable . . . 34
11.4 Failure Modes and Risk Points . . . 34

12 Reproducibility Protocol **36**
12.1 Environment Checklist . . . 36
12.2 Execution Order . . . 36

12.3 Representative Launch Commands 37

13 Limitations **38**
13.1 Measurement and Representation Limits 38
13.2 Sampling Bias and Coverage Limits 38
13.3 Temporal and Sensor Availability Constraints 38
13.4 Model Complexity Tradeoff 38

14 Future Work **39**
14.1 Short-Term Priorities 39
14.2 Medium-Term Priorities 39
14.3 Long-Term Priorities 39

15 Conclusion **40**

A Appendix A: Extended Code Snippets **41**
A.1 A.1 Earth Engine Derived Feature Utilities 41
A.2 A.2 Feature Routing Configuration Example 42
A.3 A.3 Supervised Chunk Builder Example 42
A.4 A.4 Evaluation Utility Example 43

B Appendix B: Expanded Method Notes **44**
B.1 B.1 Leakage Scenarios and Defenses 44
B.2 B.2 Why Temporal Split Is Kept as Diagnostic 44
B.3 B.3 Calibration Governance 44
B.4 B.4 Suggested Figure Slots for Final Thesis Version 44

C Appendix C: End-to-End Checklist **46**

D Appendix D: Suggested Writing Expansion Plan **47**

Acknowledgment of Draft Status **48**

Abstract

This document is the initial full draft of the Spain branch of the UG_SM project, built from the implementation and findings summarized in the project notebooks and interim technical notes. The central objective is to estimate near-surface soil moisture with improved spatial detail and improved station-level generalization by combining in situ measurements from the REMEDHUS network with multi-source Earth observation and meteorological predictors. The report is intentionally expansive and method-oriented: it explains data provenance, quality control decisions, model design, optimization strategy, evaluation rigor, and operational reproducibility in a form suitable for thesis/dissertation expansion.

The workflow is staged. First, a baseline tabular pipeline establishes data integrity and performance reference using residual XGBoost. Second, a final deep-learning pipeline introduces spatial patches, temporal sequence encoding, structured feature routing, and post-hoc calibration. The strict unseen-station score reaches the target level, with substantial improvement over baseline. This report also includes implementation-oriented code snippets so that scientific claims are traceable to concrete engineering procedures.

Executive Summary

The Spain project was selected as a high-value test bed for regional soil moisture downscaling because the REMEDHUS network provides suitable station density, long records, and meaningful hydro-climatic variation for machine learning. A two-stage strategy was adopted:

1. Build and validate a robust baseline with explicit quality checks and station-split evaluation.
2. Scale to a patch-based multimodal deep model with explicit spatial and temporal context.

Key baseline result:

- RMSE: 0.0830 m^3/m^3
- R^2: 0.3619

Key final result (selected strict protocol):

- Unseen-station R^2: 0.7016
- Unseen-station NRMSE: 6.21%

Compared with the baseline, the final system adds three critical capabilities: (i) local spatial neighborhood learning via convolutional branches, (ii) short-term hydrologic memory via LSTM sequence encoding, and (iii) residual learning around a physically meaningful microwave anchor with calibration correction. The result is a stronger and more deployment-relevant model for sites not present in training.

Chapter 1

Project Motivation and Scope

1.1 Why Spain Was Chosen

Spain is a strong study region for supervised geospatial modeling of surface soil moisture because it combines station quality, environmental diversity, and practical data availability. In this project, the REMEDHUS network is central. It offers geographically coherent station placement within an agriculturally relevant region, where seasonal wetting and drying dynamics are pronounced enough to support model learning but challenging enough to expose model weaknesses.

The selection was not only about data availability. It was about scientific leverage: the region is suitable for testing whether a model can transfer to unseen stations while preserving physically meaningful behavior. A domain where both climatic seasonality and land-surface heterogeneity are present is ideal for comparing pointwise baselines to spatial-temporal deep networks.

1.2 Problem Definition

Let the true daily near-surface volumetric soil moisture at station i and day t be $y_{i,t}$. Let $a_{i,t}$ denote the center-pixel SMAP-based anchor estimate. The modeling objective is to learn a function f such that

$$\hat{y}_{i,t} = a_{i,t} + f(\mathbf{x}_{i,t}^{\text{dyn}}, \mathbf{x}_{i}^{\text{stat}}, \mathbf{s}_{i,t}), \tag{1.1}$$

where:

- $\mathbf{x}_{i,t}^{\text{dyn}}$ represents dynamic multi-sensor variables,
- $\mathbf{x}_{i}^{\text{stat}}$ represents static soil and terrain attributes,
- $\mathbf{s}_{i,t}$ represents short sequence history features.

In residual form, the target is

$$r_{i,t} = y_{i,t} - a_{i,t}, \tag{1.2}$$

and the model predicts $\hat{r}_{i,t}$, then reconstructs $\hat{y}_{i,t} = a_{i,t} + \hat{r}_{i,t}$.

1.3 Scope Boundaries

This draft focuses on the Spain pipeline and excludes cross-country transfer experiments, global gridded deployment, and physics-constrained PDE coupling. The report covers:

- in situ acquisition and QC,
- Earth Engine feature stack design,
- baseline benchmarking,
- deep model architecture and training,
- calibration and evaluation,
- reproducibility and implementation details.

Chapter 2

Data Sources and Provenance

2.1 In Situ Labels: ISMN REMEDHUS

Ground truth originates from the International Soil Moisture Network package for Spain, specifically REMEDHUS stations. The pipeline parses CEOP-like station files and enforces strict quality filters. Measurements are restricted to the 0–5 cm layer to match target surface products.

The station files encode:

- station coordinates and elevation,
- depth interval,
- date-time records,
- measured value,
- ISMN quality and provider-level flags.

The project uses quality-flagged "good" rows only and converts timestamps to daily aggregates.

2.2 Satellite and Reanalysis Predictors

The predictor inventory combines optical, radar, thermal, passive microwave, meteorology, and static terrain/soil layers. The major sources are:

- Sentinel-2 SR harmonized (vegetation/water indices in baseline),
- Sentinel-1 GRD (VV, VH, ratio, incidence angle),
- MODIS LST and LAI,

- MODIS NDVI and albedo (in final stack),
- SMAP L4 soil moisture,
- ERA5-Land daily fields (precipitation, temperatures, derived variables),
- SoilGrids texture fractions (0–5 cm),
- NASADEM terrain layers.

The value of this mix is complementary sensing physics: radar and microwave respond differently to soil moisture and roughness; thermal and vegetation channels provide coupling context; static soil/terrain factors explain persistent local response characteristics.

2.3 Quality Constraints and Alignment Logic

Three constraints are critical:

1. Label trustworthiness: retain only high-quality in situ observations.
2. Temporal consistency: aggregate to daily support and align with daily/near-daily predictors.
3. Row-preservation robustness: prevent silent sample loss from sparse revisit and masking gaps.

The third point is especially important in geospatial ETL: image masking can reduce sampled rows in ways that look like random data loss but are systematic in cloudy or low-coverage periods.

Chapter 3

Study Area, Temporal Scope, and Sampling Geometry

3.1 Baseline Scope

The baseline notebook operates over a narrower period and broad AOI bounding box:

- AOI: $[-5.6, 41.0, -4.6, 41.8]$,
- start: 2019-03-01,
- end: 2020-02-29.

3.2 Final Scope

The final deep-learning notebook broadens temporal coverage and uses patch geometry:

- AOI: $[-5.73, 41.09, -5.09, 41.54]$,
- start: 2015-07-01,
- end: 2025-01-01,
- CRS: EPSG:32632,
- nominal scale: 1000 m,
- patch size: 13×13.

3.3 Patch-Centric Sampling

For each station-day sample, the final pipeline extracts a neighborhood patch from each channel. This creates an image-like tensor where local gradients and mixed-pixel context can be learned by convolutional layers. A radius of 6 pixels in each direction yields a 13×13 neighborhood around the station center.

Chapter 4

Baseline Pipeline (get_spain_data.ipynb)

4.1 Baseline Role in the Project

The baseline is not a throwaway model. It is a diagnostic and scientific anchor that validates the entire data chain and establishes a lower bound for expected performance. A reliable baseline provides confidence that deep-model gains are real and not artifacts of ETL mismatch.

4.2 ISMN Parsing and Label Construction

The parsing logic reads metadata and measurements, applies quality filters, and aggregates to daily means. A representative implementation pattern is shown below.

```
from pathlib import Path
import pandas as pd

def load_ismn_stm(path: Path) -> pd.DataFrame:
    lines = path.read_text(errors="ignore").splitlines()
    if len(lines) < 2:
        return pd.DataFrame()

    header = lines[0].split()
    # Expected: network, station, lat, lon, elev, depth_from,
       depth_to, ...
    station = header[1]
    lat = float(header[2])
    lon = float(header[3])
    depth_from = float(header[5])
```

```
    depth_to = float(header[6])

    if not (depth_from >= 0.0 and depth_to <= 0.05):
        return pd.DataFrame()

    rec = pd.read_csv(
        path,
        skiprows=1,
        delim_whitespace=True,
        header=None,
        usecols=[0, 1, 2, 3],
        names=["date", "time", "sm", "flag"],
    )
    rec = rec[rec["flag"] == "G"].copy()
    rec["timestamp"] = pd.to_datetime(rec["date"] + " " + rec["
       time"], errors="coerce")
    rec["sm"] = pd.to_numeric(rec["sm"], errors="coerce")
    rec = rec.dropna(subset=["timestamp", "sm"])

    rec["point_id"] = station
    rec["lat"] = lat
    rec["lon"] = lon
    return rec[["point_id", "lat", "lon", "timestamp", "sm"]]

def build_daily_labels(stm_files):
    parts = [load_ismn_stm(Path(f)) for f in stm_files]
    df = pd.concat([p for p in parts if not p.empty],
       ignore_index=True)
    df["day"] = df["timestamp"].dt.floor("D")
    return (
        df.groupby(["point_id", "lat", "lon", "day"], as_index=
           False)["sm"]
          .mean()
          .rename(columns={"day": "timestamp"})
    )
```

Listing 4.1: Representative ISMN parser for 0–5 cm good-quality rows

The baseline run confirms approximately nineteen valid files after strict depth and quality filtering in the chosen period setup.

4.3 Feature Sampling from Earth Engine

The baseline assembles a daily per-station feature table. Robustness measures include:

- buffered windows for sparse revisit sensors,
- deterministic fallback fills for missing image windows,
- explicit unmask behavior before station sampling.

Representative Earth Engine-side logic is summarized below.

```
import ee

def safe_mean(ic, start, end, bands, fallback):
    img = ic.filterDate(start, end).select(bands).mean()
    return ee.Image(ee.Algorithms.If(
        img.bandNames().size().gt(0),
        img,
        ee.Image.constant(fallback).rename(bands)
    ))

def build_feature_image(day):
    d0 = ee.Date(day)
    d1 = d0.advance(1, "day")

    s2 = safe_mean(s2_ic, d0.advance(-5, "day"), d1, ["B8", "B4",
        "B11"], [0, 0, 0])
    ndvi = s2.normalizedDifference(["B8", "B4"]).rename("NDVI")
    ndwi = s2.normalizedDifference(["B8", "B11"]).rename("NDWI")

    s1 = safe_mean(s1_ic, d0.advance(-6, "day"), d1, ["VV", "VH"
       ], [0, 0])
    vv = s1.select("VV")
    vh = s1.select("VH")
    ratio = vv.divide(vh.abs().add(1e-6)).rename("VV_VH_Ratio")

    smap = safe_mean(smap_ic, d0, d1, ["sm_surface"], [0]).rename
       ("SMAP")
    era = safe_mean(era5_ic, d0, d1, ["tp", "t2m", "d2m"], [0, 0,
        0])

    return (ndvi.addBands([ndwi, vv, vh, ratio, smap, era])
                .unmask(0)
```

```
29                 .set("day", d0.format("YYYY-MM-dd")))
```

Listing 4.2: Representative Earth Engine feature image builder pattern

4.4 Merge and Sanity Diagnostics

After export, features are merged with labels on station identity and timestamp. Reported merge quality is high (99.7%), indicating coherent station/day alignment. Diagnostics then check:

- gap-coded zero prevalence,
- feature-target correlation structure,
- station-level time series realism,
- static feature variation across sites.

4.5 Baseline Residual XGBoost Modeling

The baseline uses a residual target around SMAP:

$$\Delta y = y - a, \quad \hat{y} = a + \widehat{\Delta y}. \tag{4.1}$$

Selected engineered terms include rolling precipitation and thermal deltas. A representative training pattern is:

```
import numpy as np
from xgboost import XGBRegressor
from sklearn.metrics import mean_squared_error, r2_score

train_df = df[df.point_id.isin(train_ids)].copy()
test_df  = df[df.point_id.isin(test_ids)].copy()

for part in (train_df, test_df):
    part["sm_delta"] = part["sm"] - part["SMAP"]
    part["VV_VH_Ratio"] = part["VV"] / (np.abs(part["VH"]) + 1e
        -6)

features = [
    "SMAP", "VV", "VH", "VV_VH_Ratio", "NDVI", "NDWI",
    "tp", "t2m", "d2m", "sand", "clay", "silt", "elevation"
]
```

```
16
17 model = XGBRegressor(
18     n_estimators=1200,
19     learning_rate=0.03,
20     max_depth=6,
21     subsample=0.9,
22     colsample_bytree=0.9,
23     reg_alpha=0.2,
24     reg_lambda=2.0,
25     objective="reg:squarederror",
26     random_state=42,
27 )
28
29 model.fit(
30     train_df[features],
31     train_df["sm_delta"],
32     eval_set=[(test_df[features], test_df["sm_delta"])],
33     verbose=False,
34 )
35
36 pred_delta = model.predict(test_df[features])
37 pred = pred_delta + test_df["SMAP"].values
38
39 rmse = np.sqrt(mean_squared_error(test_df["sm"], pred))
40 r2 = r2_score(test_df["sm"], pred)
41 print({"RMSE": rmse, "R2": r2})
```

Listing 4.3: Representative baseline residual XGBoost training

4.6 Baseline Outcome

The benchmark result from the baseline notebook is:

- RMSE: 0.0830 m^3/m^3
- R^2: 0.3619

This provides a meaningful but limited predictive baseline and justifies progression to a richer model family.

Chapter 5

Final Pipeline (CNN_Spain.ipynb)

5.1 Design Rationale

The final pipeline upgrades three dimensions simultaneously:

1. **Spatial context**: learn from neighborhood patches rather than single-point rows.
2. **Temporal memory**: encode short-term forcing history via sequences.
3. **Feature specialization**: route dynamic/static/temporal signals through dedicated branches.

5.2 Long-Range Label Preparation

The final setup repeats strict quality filtering but adds robust unit handling and duplicate sensor collapse before daily aggregation. This protects the label stream over a longer time horizon.

```
def prepare_labels(raw_df):
    df = raw_df.copy()
    df = df[df["flag"] == "G"].copy()
    df["timestamp"] = pd.to_datetime(df["timestamp"], errors="
       coerce")
    df["sm"] = pd.to_numeric(df["sm"], errors="coerce")
    df = df.dropna(subset=["timestamp", "sm"])

    # Optional unit safeguard for percent-like values.
    if df["sm"].mean() > 1.0:
        df["sm"] = df["sm"] / 100.0

    # Collapse duplicate sensors at exact timestamp.
```

```
    df = (
        df.groupby(["point_id", "lat", "lon", "timestamp"],
            as_index=False)["sm"]
          .mean()
    )

    df["day"] = df["timestamp"].dt.floor("D")
    daily = (
        df.groupby(["point_id", "lat", "lon", "day"], as_index=
            False)["sm"]
          .mean()
          .rename(columns={"day": "timestamp"})
    )
    return daily
```

Listing 5.1: Representative robust label preparation for long horizon

5.3 Dynamic, Meteorological, and Static Feature Stack

The final Earth Engine stack includes daily or windowed composites across multiple sensors, with derived fields such as VPD and API.

5.3.1 Derived Variables

Vapor pressure deficit (VPD):

$$\mathrm{VPD} = e_s(T) - e_a(T_d), \tag{5.1}$$

where T is air temperature and T_d dewpoint. A common approximation is:

$$e_s(T) = 0.6108 \exp\left(\frac{17.27T}{T + 237.3}\right). \tag{5.2}$$

Antecedent precipitation index (API):

$$\mathrm{API}_t = P_t + \lambda \cdot \mathrm{API}_{t-1}, \quad 0 < \lambda < 1. \tag{5.3}$$

5.3.2 Patch Tensor Construction

For each day image, each station point is transformed into local arrays using neighborhood operators. The resulting tensors satisfy:

$$\mathbf{x}_{i,t} \in \mathbb{R}^{C \times 13 \times 13}. \tag{5.4}$$

```python
def to_patch_array(image, radius=6):
    kernel = ee.Kernel.square(radius=radius, units="pixels",
       normalize=False)
    return image.neighborhoodToArray(kernel)

def sample_station_patches(image, station_fc, scale=1000):
    arr = to_patch_array(image, radius=6)
    sampled = arr.sampleRegions(
        collection=station_fc,
        properties=["point_id", "timestamp"],
        scale=scale,
        geometries=False,
    )
    return sampled
```

Listing 5.2: Representative neighborhood-to-array sampling logic

5.4 TFRecord Export and PyTorch Ingestion

Patch samples are exported as TFRecord and consumed by a custom iterable dataset. The ingestion layer validates shape consistency and emits tensors used by downstream split and sequence construction.

```python
import torch
from torch.utils.data import IterableDataset

class PatchTFRecordDataset(IterableDataset):
    def __init__(self, tfrecord_paths, channel_count, patch_size
       =13):
        self.paths = tfrecord_paths
        self.channel_count = channel_count
        self.patch_size = patch_size

    def parse_record(self, rec):
        x = rec["x"]  # expected flattened float array
        pid = rec["point_id"]
        ts = rec["timestamp"]
        x = torch.tensor(x, dtype=torch.float32)
        x = x.view(self.channel_count, self.patch_size, self.
           patch_size)
        return x, pid, ts
```

```
17
18      def __iter__(self):
19          for path in self.paths:
20              for rec in iterate_tfrecord(path):
21                  try:
22                      yield self.parse_record(rec)
23                  except Exception:
24                      continue
```

Listing 5.3: Representative TFRecord iterable dataset sketch

5.5 Normalization and Seasonal Channels

Seasonal channels are appended:

$$\text{Day_Sin} = \sin\left(2\pi\frac{d}{365}\right), \quad \text{Day_Cos} = \cos\left(2\pi\frac{d}{365}\right), \tag{5.5}$$

where d is day-of-year.

Normalization is computed from training stations only:

$$\tilde{x}_c = \frac{x_c - \mu_c^{\text{train}}}{\sigma_c^{\text{train}} + \epsilon}. \tag{5.6}$$

This avoids station leakage.

Chapter 6

Model Architecture: MultiModalSoilNet

6.1 Overview

The final architecture is a multi-branch residual predictor. Inputs are routed by semantics rather than blended blindly:

- dynamic channels → CNN branch,
- static channels → center-pixel MLP branch,
- temporal sequence channels → LSTM branch.

Fusion concatenates branch embeddings and center features, then predicts residual correction.

6.2 Branch A: Dynamic Spatial CNN

The CNN branch learns local motifs from 13×13 dynamic patches. Adaptive pooling compresses learned activation maps to a compact embedding.

```
import torch.nn as nn

class DynamicCNN(nn.Module):
    def __init__(self, in_ch, p_drop=0.15):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(in_ch, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
```

```
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.Dropout2d(p_drop),
            nn.Conv2d(64, 64, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.AdaptiveAvgPool2d((1, 1)),
        )

    def forward(self, x):
        z = self.net(x)
        return z.flatten(1)
```

Listing 6.1: Representative dynamic CNN branch

6.3 Branch B: Static Center MLP

Static channels are sampled at the center pixel and passed through a compact fully connected encoder. This allows soil texture and terrain effects to influence prediction without dominating convolutional dynamics.

```
class StaticMLP(nn.Module):
    def __init__(self, in_dim, hid=32, p_drop=0.1):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, hid),
            nn.ReLU(inplace=True),
            nn.Dropout(p_drop),
            nn.Linear(hid, hid),
            nn.ReLU(inplace=True),
        )

    def forward(self, x_center_static):
        return self.net(x_center_static)
```

Listing 6.2: Representative static center branch

6.4 Branch C: Temporal LSTM

The LSTM branch encodes short sequence histories of center-pixel hydro-meteorological variables.

```python
class TemporalEncoder(nn.Module):
    def __init__(self, in_dim, hid=48, num_layers=2, p_drop=0.15):
        super().__init__()
        self.lstm = nn.LSTM(
            input_size=in_dim,
            hidden_size=hid,
            num_layers=num_layers,
            batch_first=True,
            dropout=p_drop,
        )

    def forward(self, seq):
        _, (h_n, _) = self.lstm(seq)
        return h_n[-1]
```

Listing 6.3: Representative temporal LSTM branch

6.5 Fusion and Output Head

The final head concatenates embeddings and center vectors, then outputs one scalar residual.

```python
class MultiModalSoilNet(nn.Module):
    def __init__(self, n_dyn, n_stat, n_seq):
        super().__init__()
        self.cnn = DynamicCNN(n_dyn)
        self.stat = StaticMLP(n_stat)
        self.temporal = TemporalEncoder(n_seq)

        fusion_in = 64 + n_dyn + n_stat + 32 + 48
        self.head = nn.Sequential(
            nn.Linear(fusion_in, 96),
            nn.ReLU(inplace=True),
            nn.Dropout(0.2),
            nn.Linear(96, 48),
            nn.ReLU(inplace=True),
            nn.Linear(48, 1),
        )

```

```
    def forward(self, x_dyn_patch, x_dyn_center, x_stat_center,
        seq):
        z_sp = self.cnn(x_dyn_patch)
        z_st = self.stat(x_stat_center)
        z_tm = self.temporal(seq)
        z = torch.cat([z_sp, x_dyn_center, x_stat_center, z_st,
            z_tm], dim=1)
        return self.head(z)
```

Listing 6.4: Representative fusion model

Chapter 7

Sequence Construction and Data Contracts

7.1 Tensor Contracts

Training inputs follow strict shape definitions:

- spatial tensor: $\mathbf{x} \in \mathbb{R}^{B \times C \times 13 \times 13}$,
- temporal tensor: $\mathbf{s} \in \mathbb{R}^{B \times L \times F}$,
- residual target: $\mathbf{r} \in \mathbb{R}^{B \times 1}$.

Maintaining explicit contracts reduces silent bugs in multi-branch models.

7.2 Station-Aware Chronological Sequence Builder

Sequences are built per station in time order, with left-padding when sequence history is short.

```
from collections import defaultdict, deque
import numpy as np

def build_sequences(samples, seq_len, feat_cols):
    # samples expected sorted by (point_id, timestamp)
    buffers = defaultdict(lambda: deque(maxlen=seq_len))
    seq_out = []

    zero = np.zeros(len(feat_cols), dtype=np.float32)

    for row in samples:
        pid = row["point_id"]
```

```
        v = np.asarray([row[c] for c in feat_cols], dtype=np.
            float32)
        buffers[pid].append(v)

        hist = list(buffers[pid])
        if len(hist) < seq_len:
            hist = [zero] * (seq_len - len(hist)) + hist

        seq_out.append(np.stack(hist, axis=0))

    return np.stack(seq_out, axis=0)
```

Listing 7.1: Representative sequence builder

This logic preserves causal ordering and avoids inter-station leakage.

Chapter 8

Training Objective, Optimization, and Tuning

8.1 Robust Weighted Huber Loss

For prediction error $e = \hat{r} - r$ and threshold δ, Huber loss is:

$$\ell_\delta(e) = \begin{cases} \frac{1}{2}e^2, & |e| \le \delta \\ \delta\left(|e| - \frac{1}{2}\delta\right), & |e| > \delta \end{cases} \tag{8.1}$$

Weighted objective:

$$\mathcal{L} = \frac{1}{N}\sum_{n=1}^{N} w_n \ell_\delta(e_n), \tag{8.2}$$

where w_n downweights extreme residual examples.

8.2 Outlier Trimming by Quantiles

Residual targets can be trimmed using training quantiles:

$$r \in [q_{\text{low}}, q_{\text{high}}]. \tag{8.3}$$

Trimming is configurable and tuned; aggressive trimming can remove real extremes, while weak trimming can overfit noise spikes.

8.3 Optimization Stack

Key mechanics:

- AdamW optimizer,

- gradient clipping,
- optional automatic mixed precision,
- early stopping on validation R^2,
- batch-level weighting and robust loss.

```python
import torch
from torch.cuda.amp import autocast, GradScaler

def train_loop(model, loader, optimizer, delta=0.05, use_amp=True):
    model.train()
    scaler = GradScaler(enabled=use_amp)
    meter = 0.0

    for batch in loader:
        x_patch, x_dyn_c, x_stat_c, seq, y_res, w = batch
        optimizer.zero_grad(set_to_none=True)

        with autocast(enabled=use_amp):
            pred = model(x_patch, x_dyn_c, x_stat_c, seq).squeeze(1)
            err = pred - y_res
            abs_err = err.abs()
            huber = torch.where(
                abs_err <= delta,
                0.5 * err * err,
                delta * (abs_err - 0.5 * delta),
            )
            loss = (w * huber).mean()

        scaler.scale(loss).backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        scaler.step(optimizer)
        scaler.update()

        meter += float(loss.detach().cpu())

    return meter / max(len(loader), 1)
```

Listing 8.1: Representative training loop skeleton

8.4 Hyperparameter Search Program

Tuning follows a staged sequence rather than one-pass random search:

1. baseline configuration,
2. Optuna station-split trials,
3. rerun with best trial,
4. polishing run,
5. aggressive refinement profile,
6. temporal fallback only if target unmet.

This staged strategy improves stability of the selected model and helps avoid over-committing to fragile trial outliers.

Chapter 9

Evaluation Protocol and Scientific Rigor

9.1 Primary vs Secondary Split

Two split families are reported:

- station split (primary): test stations unseen in training,
- temporal split (secondary): future times on known stations.

Station split is stricter and better aligned with deployment transfer.

9.2 Metrics

Reported metrics include:

- coefficient of determination R^2,
- RMSE,
- normalized RMSE (NRMSE).

Definitions:

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{n=1}^{N}(y_n - \hat{y}_n)^2}, \tag{9.1}$$

$$R^2 = 1 - \frac{\sum_n (y_n - \hat{y}_n)^2}{\sum_n (y_n - \bar{y})^2}, \tag{9.2}$$

$$\mathrm{NRMSE} = \frac{\mathrm{RMSE}}{\max(y) - \min(y)}. \tag{9.3}$$

9.3 Calibration Layer

After raw residual reconstruction, a quadratic calibrator adjusts systematic bias:

$$\hat{y} = c_0 + c_1 a + c_2 r + c_3 a r + c_4 a^2 + c_5 r^2, \tag{9.4}$$

where a is anchor and r is predicted residual. Coefficients are fit with ridge regularization.

```python
import numpy as np
from sklearn.linear_model import Ridge

def poly2_features(anchor, resid):
    return np.column_stack([
        np.ones_like(anchor),
        anchor,
        resid,
        anchor * resid,
        anchor * anchor,
        resid * resid,
    ])

def fit_calibrator(a_train, r_train, y_train, alpha=1e-2):
    X = poly2_features(a_train, r_train)
    reg = Ridge(alpha=alpha, fit_intercept=False)
    reg.fit(X, y_train)
    return reg

def apply_calibrator(cal, a, r):
    X = poly2_features(a, r)
    return cal.predict(X)
```

Listing 9.1: Representative quadratic calibration fit/apply

Chapter 10

Results

10.1 Baseline Results

Baseline performance from the Spain baseline notebook:

- RMSE: 0.0830 m^3/m^3
- R^2: 0.3619

10.2 Final Model Results

Final selected strict split result:

- split type: station,
- best validation R^2: 0.7016,
- anchor R^2: 0.1873,
- NRMSE: 6.21%,
- selected best epoch: 10.

Additional diagnostic result:

- temporal split R^2: 0.8656,
- temporal split NRMSE: 3.53%.

10.3 Interpretation

The large gain from baseline to final strict station split supports the central claim that patch-level spatial context, sequence memory, and feature routing materially improve transfer performance. The temporal split being higher is expected because station identity and static context are less novel.

10.4 Comparative Summary

Table 10.1: Spain baseline vs final summary

Model	Split	R^2	Error Metric
Residual XGBoost baseline	Station	0.3619	RMSE = 0.0830
MultiModalSoilNet (selected)	Station	0.7016	NRMSE = 6.21%
MultiModalSoilNet (diagnostic)	Temporal	0.8656	NRMSE = 3.53%

Chapter 11

Detailed Discussion

11.1 Why Residual Learning Helps

Residual learning around a physically meaningful anchor simplifies the function class the model must represent. Instead of recreating the entire moisture signal from heterogeneous predictors, the model focuses on correcting known anchor biases. This lowers optimization burden and often yields better calibration behavior.

11.2 Why Explicit Routing Helps

Feeding all channels through a single branch can cause representational imbalance. Static features can dominate if not controlled, and sequence effects can vanish if treated as static covariates. Explicit dynamic/static/temporal routing reduces this interference.

11.3 Why Station Split Is Non-Negotiable

Geospatial leakage can overstate performance. If train and test share station identity, models can encode persistent station-specific signatures. Station-split evaluation is therefore required for realistic transfer claims.

11.4 Failure Modes and Risk Points

Key risk points in this type of pipeline are:

- hidden row loss during image sampling,
- misaligned timestamp conventions,
- leakage via normalization or split strategy,

- sequence contamination across station boundaries,
- overfitting from aggressive post-hoc calibration.

The implementation actively addresses each of these through checks and design constraints.

Chapter 12

Reproducibility Protocol

12.1 Environment Checklist

Recommended reproducibility checklist:

1. lock notebook package versions,
2. preserve train/test station IDs,
3. preserve sequence length and feature order,
4. recompute normalization on train split only,
5. preserve anchor and calibration procedures,
6. log random seeds for data and model pipelines.

12.2 Execution Order

1. parse ISMN station files and build daily labels,
2. export baseline table and verify merge diagnostics,
3. run baseline benchmark,
4. build final patch stack and export TFRecords,
5. ingest, cache, augment seasonal channels,
6. split data (station primary), normalize,
7. build sequences and supervised chunks,
8. train/tune model,

9. fit/apply calibration,

10. evaluate and archive outputs.

12.3 Representative Launch Commands

```
# 1) Install and verify dependencies (example)
pip install -U numpy pandas xgboost torch torchvision optuna
   scikit-learn

# 2) Run baseline notebook (if converted to script)
python run_baseline_spain.py

# 3) Run patch export workflow (Earth Engine auth required)
python export_spain_patches.py

# 4) Train final multimodal model
python train_spain_multimodal.py --split station --target-r2 0.70

# 5) Evaluate and summarize
python evaluate_spain.py --model-path outputs/best_model.pt
```

Listing 12.1: Representative environment and workflow commands

Chapter 13

Limitations

13.1 Measurement and Representation Limits

Surface soil moisture is influenced by scale mismatch between point observations and kilometer-scale pixels. Even high-quality labels can contain representativeness mismatch in mixed land-cover pixels.

13.2 Sampling Bias and Coverage Limits

Station locations are not random and may emphasize specific land management regimes. Learned behavior may degrade in out-of-distribution areas.

13.3 Temporal and Sensor Availability Constraints

Cloud, revisit intervals, and product latency can reduce effective daily signal availability, increasing dependence on fallback logic.

13.4 Model Complexity Tradeoff

Deep multimodal systems improve accuracy but increase implementation complexity, compute cost, and operational monitoring burden.

Chapter 14

Future Work

14.1 Short-Term Priorities

- Add uncertainty quantification (ensembles or evidential heads).
- Perform ablations for each branch and each feature family.
- Add station-wise reliability and drift dashboards.
- Validate calibration stability over seasonal partitions.

14.2 Medium-Term Priorities

- Extend to multi-country transfer with strict leave-region-out protocols.
- Add physically inspired constraints to reduce implausible dynamics.
- Explore transformer-style temporal encoders and lightweight deployment variants.

14.3 Long-Term Priorities

- Build a reproducible benchmark suite across climate zones.
- Integrate uncertainty-aware decision support for irrigation and drought analytics.
- Bridge to operational NRT pipelines with robust monitoring and retraining triggers.

Chapter 15

Conclusion

This Spain draft documents a complete end-to-end progression from robust baseline to strong station-generalization deep model. The final system exceeds the project target on strict unseen-station evaluation and demonstrates that structured multimodal learning with residual anchoring can materially improve surface soil moisture prediction quality.

Beyond headline metrics, the core contribution is methodological discipline: quality-controlled labels, leakage-aware splits, explicit routing, robust optimization, and reproducibility contracts. These design choices make the pipeline a viable foundation for subsequent scientific refinement and operational adaptation.

Appendix A

Appendix A: Extended Code Snippets

A.1 A.1 Earth Engine Derived Feature Utilities

```
import ee

def saturation_vapor_pressure_celsius(tc):
    # tc in Celsius
    return ee.Image(0.6108).multiply(
        (ee.Image(17.27).multiply(tc).divide(tc.add(237.3))).exp()
    )

def make_vpd_from_era5(era_img):
    t2m_c = era_img.select("t2m").subtract(273.15)
    d2m_c = era_img.select("d2m").subtract(273.15)
    es = saturation_vapor_pressure_celsius(t2m_c)
    ea = saturation_vapor_pressure_celsius(d2m_c)
    return es.subtract(ea).max(0).rename("VPD")

def make_api(precip_series, decay=0.85):
    api = ee.Image.constant(0)
    out = []
    for img in precip_series:
        api = ee.Image(img).add(api.multiply(decay)).rename("API")
        out.append(api)
    return out
```

Listing A.1: Representative ERA5-derived variable helpers

A.2 A.2 Feature Routing Configuration Example

```
routing = {
    "dynamic": [
        "NDVI", "VV", "VH", "VV_VH_Ratio", "LST", "SMAP",
        "Albedo", "precip", "VPD", "API", "Day_Sin", "Day_Cos"
    ],
    "static": ["clay", "sand", "silt", "elevation", "slope", "
       aspect"],
    "temporal": ["SMAP", "precip", "VPD", "API"],
}

def channel_indices(all_channels, names):
    idx = []
    for n in names:
        idx.append(all_channels.index(n))
    return idx

dyn_idx = channel_indices(all_channels, routing["dynamic"])
stat_idx = channel_indices(all_channels, routing["static"])
tmp_idx = channel_indices(all_channels, routing["temporal"])
```

Listing A.2: Representative channel routing map

A.3 A.3 Supervised Chunk Builder Example

```
import numpy as np

def make_supervised_chunk(rows, q_low=None, q_high=None):
    y = np.asarray([r["sm"] for r in rows], dtype=np.float32)
    a = np.asarray([r["SMAP_center"] for r in rows], dtype=np.
       float32)
    residual = y - a

    keep = np.ones_like(residual, dtype=bool)
    if q_low is not None and q_high is not None:
```

```
        lo = np.quantile(residual, q_low)
        hi = np.quantile(residual, q_high)
        keep = (residual >= lo) & (residual <= hi)

    w = 1.0 / (np.abs(residual) + 0.02)
    w = w / (w.mean() + 1e-8)

    return {
        "x": np.asarray([r["x"] for r in rows], dtype=np.float32)
           [keep],
        "seq": np.asarray([r["seq"] for r in rows], dtype=np.
           float32)[keep],
        "y_res": residual[keep],
        "anchor": a[keep],
        "w": w[keep],
    }
```

Listing A.3: Representative supervised chunk construction

A.4 A.4 Evaluation Utility Example

```
import numpy as np
from sklearn.metrics import r2_score, mean_squared_error

def eval_metrics(y_true, y_pred):
    rmse = float(np.sqrt(mean_squared_error(y_true, y_pred)))
    r2 = float(r2_score(y_true, y_pred))
    nrmse = float(rmse / (np.max(y_true) - np.min(y_true) + 1e-8)
       )
    return {"RMSE": rmse, "R2": r2, "NRMSE": nrmse}
```

Listing A.4: Representative evaluation helper

Appendix B

Appendix B: Expanded Method Notes

B.1 B.1 Leakage Scenarios and Defenses

Common leakage scenarios in geospatial ML include shared station identity across splits, temporal overlap leakage, and global normalization statistics. The Spain final pipeline addresses these with station-centric split definitions and train-only normalization.

B.2 B.2 Why Temporal Split Is Kept as Diagnostic

Temporal split remains useful for checking model stability under known-station forecasting conditions and for identifying whether gains are mostly from static memorization or genuine transfer. It is not used as the final headline criterion.

B.3 B.3 Calibration Governance

Calibration must be fit on training/validation partitions only and then frozen before test use. Re-fitting on test predictions invalidates evaluation claims.

B.4 B.4 Suggested Figure Slots for Final Thesis Version

For the next iteration, include:

1. Spain AOI and station map,
2. feature timeline coverage diagnostics,

3. baseline feature importance,
4. predicted vs observed scatter for each split,
5. residual histogram pre- and post-calibration,
6. station-wise performance distribution,
7. seasonal performance stratification.

Appendix C

Appendix C: End-to-End Checklist

Step	Procedure
1	Acquire and inspect ISMN REMEDHUS package files.
2	Parse headers and records from station files.
3	Keep only depth 0–5 cm observations.
4	Keep only good quality flags for labels.
5	Aggregate records to daily station labels.
6	Initialize Earth Engine and define AOI/date ranges.
7	Build dynamic collections (optical, radar, thermal, microwave, meteorology).
8	Build static soil and terrain layers.
9	Derive VPD and API terms.
10	Apply robust missing-data safeguards before sampling.
11	Export baseline table and merge with labels.
12	Run baseline diagnostics and baseline residual model.
13	Build final patch stack with neighborhood arrays.
14	Export TFRecord patches and ingest in PyTorch.
15	Cache samples and append seasonal channels.
16	Define station split and compute train-only normalization.
17	Build station-aware temporal sequences.
18	Construct supervised chunks with residual and weights.
19	Train MultiModalSoilNet with early stopping and clipping.
20	Tune hyperparameters with staged search strategy.
21	Fit and apply quadratic calibration layer.
22	Evaluate station and temporal splits, select strict result.
23	Archive metrics, split IDs, configs, and artifacts.

Appendix D

Appendix D: Suggested Writing Expansion Plan

This initial draft is intentionally dense and implementation-forward. For thesis-level finalization, expand in this order:

1. Add literature review chapter (remote sensing soil moisture downscaling, multimodal ML, residual calibration methods).
2. Add figure-rich exploratory analysis chapter with station climatology and variable distributions.
3. Add complete ablation chapter (branch removals, feature family drops, trim/no-trim, calibration/no-calibration).
4. Add uncertainty chapter (ensemble spread, confidence intervals, reliability analysis).
5. Add deployment chapter (latency, update cadence, monitoring, retraining triggers).

Acknowledgment of Draft Status

This document is an initial full technical draft generated from the implemented Spain project workflow and current project notes. It is intended as a high-detail writing base for iterative refinement, figure integration, and formal referencing in a final report submission.